

1. To write an ARM Cortex-M0 assembly program that compares a counter value with 10 and resets the counter to 0 if it is greater than 10; otherwise, it increments the counter by 1.

The aim of this program is to check the value of a counter and perform conditional operations using ARM Cortex-M0 assembly instructions.

High-level logic

```
if (counter > 10)
    counter = 0
else
    counter = counter + 1
```

ARM Cortex-M0 Assembly Program

```
PRESERVE8
THUMB
AREA |.text|, CODE, READONLY

EXPORT __main

__main
    MOVS R0, #5           ; Initialize counter (example value)

check_counter
    CMP R0, #10           ; Compare counter with 10
    BLE incr_counter      ; If counter ≤ 10, go to increment

    MOVS R0, #0           ; If counter > 10, reset counter to 0
    B counter_done        ; Skip increment

incr_counter
    ADDS R0, R0, #1       ; counter = counter + 1

counter_done
    B check_counter       ; Repeat the process (loop)

END
```

2. To implement a looping operation using ARM Cortex-M0 assembly instructions to calculate the sum of numbers from 0 to 4..

High-Level Program (C-like)

```
Total = 0;
for(i = 0; i < 5; i = i + 1)
    Total = Total + i;
```

ARM Cortex-M0 Assembly Program

```
PRESERVE8
THUMB
AREA |.text|, CODE, READONLY
EXPORT __main

__main

    MOVS R0, #0          ; Total = 0
    MOVS R1, #0          ; i = 0

loop
    ADDS R0, R0, R1      ; Total = Total + i
    ADDS R1, R1, #1      ; i = i + 1
    CMP  R1, #5          ; Compare i with 5
    BLT  loop            ; If i < 5, repeat loop

stop
    B stop               ; Infinite loop (program end)

END
```

3. To write an ARM Cortex-M0 assembly program that executes a loop five times using a loop counter.

```
PRESERVE8
THUMB
AREA |.text|, CODE, READONLY
EXPORT __main

__main

    MOVS R0, #5          ; Load loop counter = 5

loop
    SUBS R0, R0, #1      ; Decrement counter
    BNE  loop            ; If result not zero, repeat loop

stop
    B stop               ; End of program (infinite loop)

END
```

4. To write an ARM Cortex-M0 assembly program that polls a status register until bit 3 becomes 1.

To write an ARM Cortex-M0 assembly program that **continuously checks a status register and exits the loop when bit-3 becomes set (1).**

```
PRESERVE8
THUMB
AREA |.text|, CODE, READONLY
EXPORT __main

Status  EQU 0x40000000
```

```

__main
    LDR R0, =Status
    MOVS R2, #0x08

loop
    LDR R1, [R0]
    TST R1, R2
    BEQ loop

stop
    B stop
END

```

1. Load Address of Status Register

```
LDR R0, =Status
```

Status register address:

Status = 0x40000000

After execution:

R0 = 0x40000000

This means **R0 now points to the status register.**

2. Load Bit Mask

```
MOVS R2, #0x08
```

R2 = 00001000

This mask is used to **check bit-3.**

Loop Execution

3. Read Status Register

```
LDR R1, [R0]
```

Assume the status register initially contains:

R1 = 00000010

Binary:

00000010

Bit-3 = **0**

4. Test Bit Using AND Operation

```
TST R1, R2
```

TST performs:

R1 AND R2

Calculation

R1 = 00000010

R2 = 00001000

```

-----
          00000000

```

Result:

0

Zero flag **Z = 1**

5. Branch if Equal

```
BEQ loop
```

BEQ means **Branch if Zero flag = 1.**

Since result = **0**, program returns to **loop**.

So the processor **keeps checking again and again.**

Second Example (Device Ready)

Suppose the status register later becomes:

R1 = 00001010

Binary:

00001010

Bit-3 = **1**

Test Operation

```
TST R1, R2
```

Program Flow

Start

|

Load Status Address

|

Load Mask (bit3)

|

Read Status Register

|

Check bit3

|

|— bit3 = 0 → repeat loop

|

|— bit3 = 1 → exit loop

Calculation:

R1 = 00001010

R2 = 00001000

00001000

Result:

00001000 $\neq$ 0

Zero flag **Z = 0**

6. BEQ Condition

BEQ loop

Since **Z = 0**, branch **does not occur**.

Program exits the loop.

5. To write an ARM Cortex-M0 assembly program that performs multi-way branching using a branch table for values 0–3 and executes a default case for other values.

Equivalent C Program

```
switch(R0)
{
case 0: R3=10; break;
case 1: R3=20; break;
case 2: R3=30; break;
case 3: R3=40; break;
default: R3=0;
}
```

Assembly Program

```
        PRESERVE8
        THUMB
        AREA |.text|, CODE, READONLY
        EXPORT __main

__main

        MOVS R0,#2          ; Example input value (case number)

        CMP  R0,#3          ; Compare with maximum case value
        BHI  default_case   ; If R0 > 3 go to default case

        MOVS R2,#4          ; Each table entry is 4 bytes
        MULS R0,R2,R0       ; Offset = R0 × 4

        LDR  R1,=BranchTable ; Load base address of branch table
        LDR  R2,[R1,R0]     ; Load destination address
        BX   R2             ; Branch to selected case

        ALIGN 4

BranchTable
        DCD Dest0
        DCD Dest1
```

Address Offset	Destination
0	Dest0
4	Dest1
8	Dest2
12	Dest3

```

        DCD Dest2
        DCD Dest3

default_case
        MOVS R3,#0           ; Default operation
        B stop

Dest0
        MOVS R3,#10          ; Case 0 operation
        B stop

Dest1
        MOVS R3,#20          ; Case 1 operation
        B stop

Dest2
        MOVS R3,#30          ; Case 2 operation
        B stop

Dest3
        MOVS R3,#40          ; Case 3 operation

stop
        B stop               ; End of program

END

```

6. To write an ARM Cortex-M0 assembly program to copy 100 bytes of data from source address 0x00000000 to destination address 0x20000000.

To write an ARM Cortex-M0 assembly program to **copy 100 bytes from source memory to destination memory**.

```

        PRESERVE8
        THUMB
        AREA |.text|, CODE, READONLY
        EXPORT __main

__main

        LDR R0, =0x00000000   ; Source address
        LDR R1, =0x20000000   ; Destination address
        LDR R2, =100          ; Number of bytes to copy

copy_loop
        LDRB R3, [R0]         ; Read 1 byte from source
        ADDS R0, R0, #1       ; Increment source pointer
        STRB R3, [R1]         ; Write byte to destination
        ADDS R1, R1, #1       ; Increment destination pointer
        SUBS R2, R2, #1       ; Decrement counter
        BNE copy_loop         ; Repeat until counter = 0

stop
        B stop               ; End of program

END

```

7. To copy 100 bytes using register offset addressing to reduce instructions inside the loop.

Optimized Assembly Program (Register Offset Addressing)

```
PRESERVE8
THUMB
AREA |.text|, CODE, READONLY
EXPORT __main

__main

    LDR R0, =0x00000000    ; Source address
    LDR R1, =0x20000000    ; Destination address
    LDR R2, =100           ; Number of bytes

copy_loop
    SUBS R2, R2, #1        ; Decrement counter
    LDRB R4, [R0, R2]      ; Read byte from source
    STRB R4, [R1, R2]      ; Write byte to destination
    BNE copy_loop          ; Continue until all bytes copied

stop
    B stop

END
```

8. To write an ARM Cortex-M assembly program to copy 128 bytes of data using block transfer instructions (LDMIA and STMIA).

```
PRESERVE8
THUMB
AREA |.text|, CODE, READONLY
EXPORT __main

__main

    LDR r0, =0x00000000    ; Source address
    LDR r1, =0x20000000    ; Destination address
    LDR r2, =128           ; Number of bytes to copy (loop counter)

copy_loop

    LDMIA r0!, {r4-r7}     ; Read 4 words (16 bytes) and increment r0
    STMIA r1!, {r4-r7}     ; Store 4 words and increment r1

    LDMIA r0!, {r4-r7}     ; Read next 4 words
    STMIA r1!, {r4-r7}

    LDMIA r0!, {r4-r7}     ; Read next 4 words
    STMIA r1!, {r4-r7}

    LDMIA r0!, {r4-r7}     ; Read next 4 words
    STMIA r1!, {r4-r7}

    SUBS r2, r2, #64       ; Each loop copies 64 bytes
    BNE copy_loop          ; Repeat until all data copied

stop
    B stop                 ; End of program
```

END

9. To write an ARM Cortex-M0 assembly program that creates two local variables on the stack, stores values in them, reads them back, and restores the stack pointer before returning from the function.

```
PRESERVE8
THUMB
AREA |.text|, CODE, READONLY
EXPORT __main
EXPORT function1

__main
    BL function1        ; Call the function
stop
    B stop              ; End of program

;-----
; Function Definition
;-----

function1
    SUB SP, SP, #8      ; Reserve 8 bytes (2 words) for local variables

    MOVS R0, #0x12      ; Load dummy value 0x12

    STR R0, [SP, #0]    ; Store value in 1st local variable
    STR R0, [SP, #4]    ; Store value in 2nd local variable

    LDR R1, [SP, #0]    ; Read value from 1st local variable
    LDR R2, [SP, #4]    ; Read value from 2nd local variable

    ADD SP, SP, #8      ; Restore stack pointer

    BX LR              ; Return from function

END
```

Program Flow

Main Program

```
|
| BL function1
v
```

function1

```
|
Reserve stack space
Store local variables
Read local variables
Restore stack
Return (BX LR)
```

10. To perform 64-bit addition of two numbers using 32-bit registers in ARM assembly.

```
PRESERVE8
THUMB
AREA |.text|, CODE, READONLY
EXPORT __main

__main

    LDR R0, =0xFFFFFFFF      ; X_Low  (Lower 32 bits of X)
    LDR R1, =0x3333FFFF      ; X_High (Upper 32 bits of X)

    LDR R2, =0x00000001      ; Y_Low  (Lower 32 bits of Y)
    LDR R3, =0x33330000      ; Y_High (Upper 32 bits of Y)

    ADDS R0, R0, R2           ; Add lower 32 bits
    ADCS R1, R1, R3           ; Add upper 32 bits with carry

stop
    B stop

END
```

11. To perform 64-bit subtraction (X – Y) using 32-bit ARM registers.

```
PRESERVE8
THUMB
AREA |.text|, CODE, READONLY
EXPORT __main

__main

    LDR R0, =0x00000001      ; X_Low  (Lower 32 bits of X)
    LDR R1, =0x00000001      ; X_High (Upper 32 bits of X)

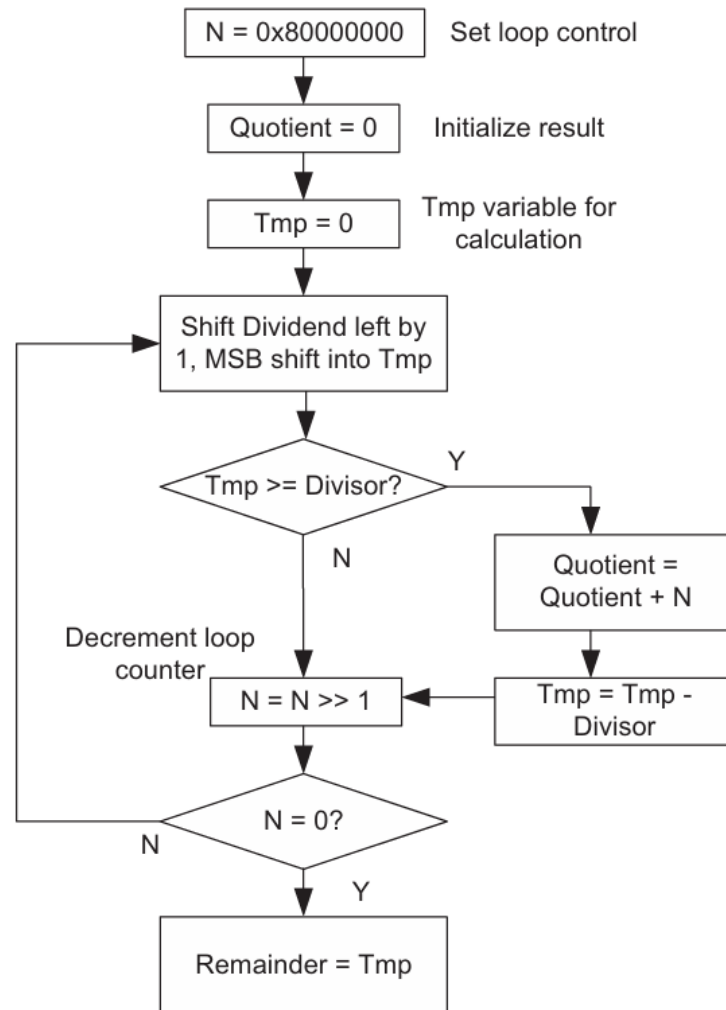
    LDR R2, =0x00000003      ; Y_Low
    LDR R3, =0x00000000      ; Y_High

    SUBS R0, R0, R2           ; Subtract lower 32 bits
    SBCS R1, R1, R3           ; Subtract upper 32 bits with borrow

stop
    B stop

END
```


12. To write an ARM Cortex-M0 assembly function to perform integer division using shift and subtraction operations.



```

PRESERVE8
THUMB

AREA    |.text|, CODE, READONLY
EXPORT  __main

__main

;-----
; Load Dividend and Divisor (example)
;-----
    MOV    R0, #13      ; Dividend = 13
    MOV    R1, #3       ; Divisor  = 3

;-----
; Call Division Function
;-----
    BL     simple_divide

; After return:
; R0 = Quotient
; R1 = Remainder
  
```

```

stop          B          stop          ; Infinite loop to view result

```

```

;-----
; Function: simple_divide
;-----
; Inputs :
;   R0 = Dividend
;   R1 = Divisor
; Outputs:
;   R0 = Quotient
;   R1 = Remainder
;-----

```

```

simple_divide

```

```

        PUSH    {R2-R4}          ; Save registers

        MOV     R2, R0            ; Copy dividend
        MOVS    R3, #1
        LSLS    R3, R3, #31      ; R3 = 0x80000000 (bit mask)

        MOVS    R0, #0            ; Quotient = 0
        MOVS    R4, #0            ; Remainder = 0

```

```

loop

```

```

        LSLS    R2, R2, #1        ; Shift dividend left
        ADCS    R4, R4, R4        ; Shift remainder + carry

        CMP     R4, R1            ; Compare remainder and divisor
        BCC     skip              ; If R4 < R1, skip subtraction
                                   ; Branch if Carry Clear

        SUBS    R4, R4, R1        ; Remainder = R4 - divisor
        ADDS    R0, R0, R3        ; Set quotient bit

```

```

skip

```

```

        LSRS    R3, R3, #1        ; Shift mask right
        BNE     loop              ; Repeat for 32 bits

        MOV     R1, R4            ; Move remainder to R1

        POP     {R2-R4}          ; Restore registers
        BX     LR                 ; Return

        END

```

```

MOV R0, #13    ; Dividend
MOV R1, #3     ; Divisor

```

Iteration 1

```

R2 = 1101 → shift → 1010
Carry = 1

```

```

R4 = 0000 → shift + carry → 0001

```

```

Compare: 1 < 3 → No subtraction
Quotient bit = 0

```

Iteration 2

R2 = 1010 → shift → 0100

Carry = 1

R4 = 0001 → shift + carry → 0011 (3)

Compare: $3 \geq 3$ → Subtract

R4 = $3 - 3 = 0$

Quotient bit = 1

Iteration 3

R2 = 0100 → shift → 1000

Carry = 0

R4 = 0000 → shift + carry → 0000

Compare: $0 < 3$ → No subtraction

Quotient bit = 0

Iteration 4

R2 = 1000 → shift → 0000

Carry = 1

R4 = 0000 → shift + carry → 0001

Compare: $1 < 3$ → No subtraction

Quotient bit = 0

Quotient bits collected:

0 1 0 0 → 4

R0 = 4 → Quotient

R1 = 1 → Remainder

How quotient is formed

When condition is true:

ADDS R0, R0, R3; R0 = R0 + current_bit_value

So quotient is built like:

Bit positions: 8 4 2 1

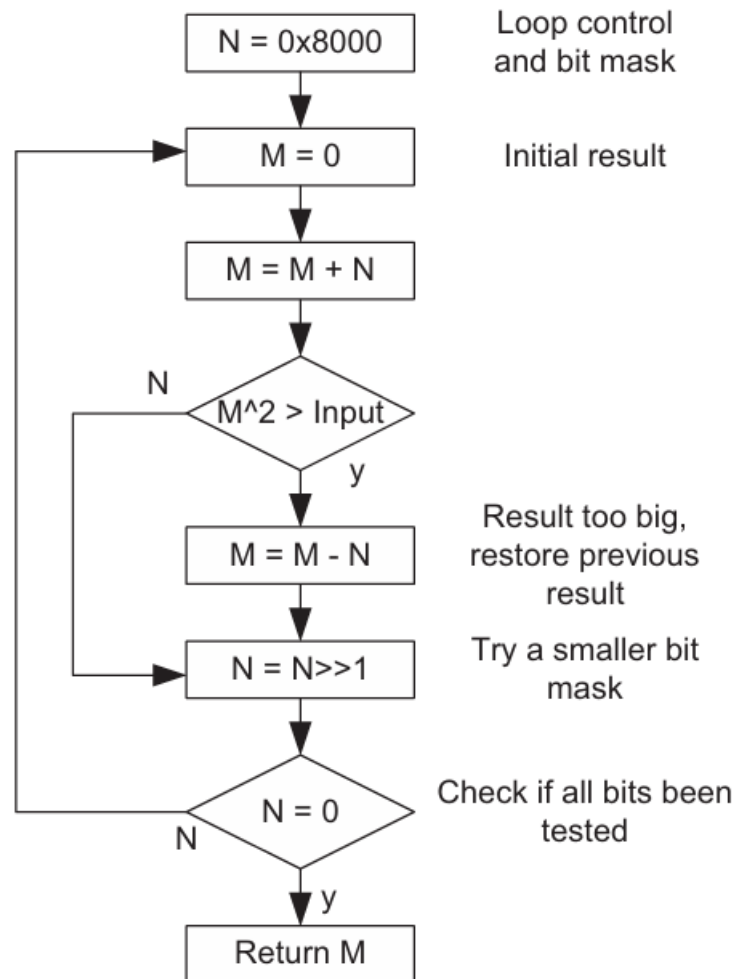
Example bits : 0 1 0 0 → 4

Register Role

R0	Input/Output (Quotient)
R1	Input/Output (Remainder)
R2	Temporary (Dividend copy)
R3	Bit mask
R4	Temporary (Remainder)

We use `PUSH {R2-R4}` to **save the original values of registers** before using them inside the function.

13. To write an ARM Cortex-M0 assembly function to calculate the integer square root of a number using iterative bit testing.



```

PRESERVE8
THUMB
AREA |.text|, CODE, READONLY
EXPORT __main
EXPORT simple_sqrt

__main
    MOVS R0, #25          ; Example input number
    BL  simple_sqrt      ; Call square root function

stop
    B stop               ; End of program

;-----
; Function : simple_sqrt
; Input   : R0 (number)
; Output  : R0 (square root)
;-----

simple_sqrt
    PUSH {R1-R3}        ; Save registers

```

```

        MOVS R1,#1          ; Initialize loop control
        LSLS R1,R1,#15      ; R1 = 0x00008000

        MOVS R2,#0          ; Result = 0

simple_sqrt_loop

        ADDS R2,R2,R1       ; M = M + N
        MOV  R3,R2          ; Copy to R3
        MULS R3,R3,R3       ; R3 = (M+N)^2

        CMP  R3,R0          ; Compare with input
        BLS  simple_sqrt_ok

        SUBS R2,R2,R1       ; Restore M if square too large

simple_sqrt_ok

        LSRS R1,R1,#1       ; N = N >> 1
        BNE  simple_sqrt_loop

        MOV  R0,R2          ; Move result to R0

        POP {R1-R3}         ; Restore registers
        BX  LR              ; Return

        END

```

Algorithm

This program uses a **bit-by-bit method (trial method)**. It builds the square root **gradually**:

1. Try adding a value (N) to result (M)
2. Check if $(M + N)^2 \leq \text{number}$
3. If yes $\rightarrow$ keep it
4. If no $\rightarrow$ remove it
5. Shift N right and repeat

```

R0 = 25          ; Input
R1 = 0x8000      ; Trial value (N)
R2 = 0           ; Result (M)

```

Loop runs **16 times** (since 16-bit precision)

When R1 = 8

M = 0

Try: $M + N = 0 + 8 = 8$
Square: $8^2 = 64$

Compare: $64 > 25$
 $\rightarrow$ Reject

$$M = 0$$

When $R1 = 4$

$$M = 0$$

$$\text{Try: } M + N = 0 + 4 = 4$$

$$\text{Square: } 4^2 = 16$$

$$\text{Compare: } 16 \leq 25$$

→ Accept

$$M = 4$$

When $R1 = 2$

$$M = 4$$

$$\text{Try: } M + N = 4 + 2 = 6$$

$$\text{Square: } 6^2 = 36$$

$$\text{Compare: } 36 > 25$$

→ Reject

$$M = 4$$

When $R1 = 1$

$$M = 4$$

$$\text{Try: } M + N = 4 + 1 = 5$$

$$\text{Square: } 5^2 = 25$$

$$\text{Compare: } 25 \leq 25$$

→ Accept

$$M = 5$$

Final Result

$$R0 = 5 \quad ; \quad \sqrt{25}$$